Assignment 4 – Ray Tune & Optuna for Transformer Translation

English to Hindi Translation Optimization

M25CSA033 Akanksha Kapil

Part 1 – Baseline Metrics

I ran the provided **en_to_hi.ipynb** notebook end-to-end on Google Colab without changing anything. The model is a standard Transformer with the following fixed hyperparameters:

Hyperparameter	Value
d_model	512
num_layers	6
num_heads	8
d_ff	2048
dropout	0.1
learning rate	1e-4
batch_size	60
optimizer	Adam

After training for 100 epochs, here are the numbers I recorded:

Metric	Baseline Value
Total Training Time	~115 minutes
Final Loss (Epoch 100)	0.0978
BLEU Score (NLTK)	71.47

Each epoch took around 69 seconds on Colab (GPU). The BLEU score of 71.47 is what I needed to match or beat in fewer epochs in Part 2.

Part 2 – Refactoring for Ray Tune & Optuna

Training Loop Changes

I created a new function **train_tune(config)** that takes a config dictionary from Ray Tune instead of using hardcoded globals. The main changes from the original **train()** function are:

- Model, optimizer, and loss function are all initialized using values from config
- Instead of printing or saving checkpoints, each epoch ends with **tune.report({"loss": avg_loss, "epoch": epoch+1})**

- Added gradient clipping (max_norm=1.0) and CosineAnnealingLR to help converge faster
- Used .reshape() instead of .view() to avoid runtime errors with non-contiguous tensors

Hyperparameters Tuned

I tuned 6 hyperparameters in total .Here is the search space I used:

Hyperparameter	Tune API	Range / Choices	Why I chose this
lr	loguniform	1e-5 to 1e-3	Biggest impact on training; log scale makes sense
batch_size	choice	32, 64, 128	Affects how noisy the gradients are
num_heads	choice	4, 8	Both divide 512 (d_model) evenly
d_ff	choice	1024, 2048	Baseline used 2048; wanted to check if 1024 works
dropout	uniform	0.1 to 0.4	Too much dropout hurts convergence
num_layers	choice	4, 6	Fewer layers = faster epochs

Sweep Setup

I used OptunaSearch with 20 trials, each capped at 30 epochs. To avoid wasting time on bad configs, I also added ASHAScheduler which kills a trial early if it isn't improving after 5 epochs. Resources were set to 0.5 GPU per trial so two trials could run in parallel on the single Kaggle T4 GPU.

Part 3 – Best Config & Final Results

Best Configuration Found

After the sweep finished, Optuna picked the following as the best config (lowest loss after 30 epochs):

Hyperparameter	Best Value	Baseline Value
lr	0.000188	0.0001
batch_size	64	60
num_heads	8	8
d_ff	1024	2048
dropout	0.1047	0.1
num_layers	6	6

The biggest difference from the baseline is d_ff dropping from 2048 to 1024. This made each epoch noticeably faster without hurting quality. The learning rate is also slightly higher (0.000188 vs 0.0001), which along with cosine LR decay helped the model converge quicker.

Final Comparison

Metric	Baseline	Best Model	Difference
Epochs	100	30	70% fewer
Training Time	~115 min	26.7 min	~4x faster
Final Loss	0.0978	0.1668	—
BLEU Score	71.47	**79.03**	**+7.56**

The best model scored **79.03 BLEU** in just 30 epochs, which is **7.56 points higher** than the 100-epoch baseline. Training time also dropped from ~115 minutes to under 27 minutes. The efficiency goal was met — baseline BLEU was matched and exceeded well within the 30-epoch limit.

Below are some sample translations from the best model to verify it's producing sensible output:

English	Hindi (predicted)
I love you.	मैं तुमसे प्यार करता हूँ।
What is your name?	तुम्हारा नाम क्या है?
How are you?	आप कैसे हो?
The weather is nice today.	आज मौसम बहुत अच्छा है।
She is a good teacher.	वह एक अच्छी अध्यापिका है।

Hugging face link : https://huggingface.co/Akanksha8822/en-to-hi-transformer

Github link: https://github.com/AkankshaKapil/MLOPS_Akanksha_M25CSA033/tree/Assignment-4